

Crescent: Stronger Privacy for Existing Credentials

Christian Paquin, Guru-Vamsi Policharla and Greg Zaverucha

DRAFT, March 26, 2025

Abstract

We describe Crescent, a construction and implementation of privacy-preserving credentials. The system works by upgrading the privacy features of existing credentials, such as JSON Web Tokens (JWTs) and Mobile Driver’s License (mDL) and as such does not require a new party to issue credentials. By using zero-knowledge proofs of possession of these credentials, we can add privacy features such as selective disclosure and unlinkability, without help from credential issuers. The system has practical performance, offering fast proof generation and verification times (tens of milliseconds) after a once-per-credential setup phase. We give demos for two practical scenarios, proof of employment for benefits eligibility (based on an employer-issued JWT), and online age verification (based on an mDL). We provide an open-source implementation to enable further research and experimentation.

This paper is a draft describing our work, aiming to include enough material to describe the functionality, and some details of the internals of our new library, available at <https://github.com/microsoft/crescent-credentials>.

1 Introduction

Digital Identity and Mobile Wallets Long-lived digital credentials for identity are becoming more common, with primary examples being the mobile Driver’s License (mDL), Selective Disclosure JSON Web Tokens (SD-JWT) [FYC24], and W3C Verifiable Credentials (VCs). The mDL standard is already used by eleven US states to issue residents a digital version of their driver’s license, with 14 other states working to implement it. Mobile wallets are also increasingly expanding beyond payments to include credentials relating to employment (e.g., Microsoft Entra Verified ID¹ and LinkedIn’s Place of Work Verification²) and healthcare (e.g. Smart Health Cards³ for proof of vaccination). The European digital identity (eID) regulation aims to provide identification and authentication by means of a personal digital wallet on a mobile phone [Eur]. This is exciting, since it will enable better authentication in new online scenarios, but we are also concerned with the privacy implications of these new technologies.

The mDL and SD-JWT did introduce some limited privacy protections. Basically the credential is a signed list of randomized commitments to the attributes, that the user can selectively open when presenting the credential. While selective disclosure (SD) is welcome and important, every use of the credential may be correlated (by using the signature as a handle). The problem is compounded when groups of verifiers collude and share information.

VCs define an abstract credential representation that can be realized using various underlying credential types that support SD. In addition to SD-JWT, they can also support schemes such as CL or BBS-based anonymous credentials, that provide both unlinkability and SD. Even once these schemes are standardized (e.g., BBS is going through CFRG standardization [LKW25]), many extensions and integration profiles will need to be written, resulting in significant changes in today’s issuance infrastructure, which will complicate and/or deter adoption of these technologies. It is therefore likely that we’ll have entrenched deployments with limited privacy for the foreseeable future.

In the academic literature there is a folklore construction of anonymous credentials from traditional credentials; given a sufficiently general zero-knowledge proof system, the credential holder simply proves

¹<https://learn.microsoft.com/en-us/entra/verified-id/>

²<https://learn.microsoft.com/en-us/entra/verified-id/linkedin-employment-verification>

³<https://smarthealth.cards/en/>

knowledge of a valid credential. More concretely, for a JWT, (which is a list of attributes encoded as JSON with an appended signature), the user would prove that 1) some private base64 data is signed by hashing it, 2) that the signature verification equation holds, 3) the signed data decodes to JSON, 4) the encoded expiry date is shown to be in the future, and 5) the attributes are revealed (or proven to satisfy some predicates).

Perhaps surprisingly, proving knowledge of such complex credentials has been shown to be approaching practicality. With recent advances on the engineering side of SNARKs (driven in part by academic research, but also by large investments in the blockchain space), these approaches are constantly getting more practical. We mention two prominent examples here and discuss related work in more detail in Section 5. There was the Cinderella paper [DFKP16] that created proofs of X.509 certificates (including chain building), and more recently the zk-creds [RWGM23] work builds a new credential system that proves knowledge of signed passport data as part of vetting users during issuance.

The most significant limitation of these works, common to many applications of zk-SNARKs is that the time required to generate a proof is high: tens of seconds. Flexibility is also limited since changing the statement to prove (e.g., selective disclosure with a different set of attributes) requires a change to the circuit. Furthermore, the performance of specific examples does not generalize all that well, since the cost of SNARKs is hard to predict, for instance a change in the issuer’s signing algorithm could give wildly different performance. From the Cinderella or zk-creds effort, it is therefore not obvious how expensive an mDL proof would be. Without benchmarks for commonly used credentials, proposing use of this idea is blocked since an informed consideration of costs is essential (for any technology).

Crescent: An anonymous credential system built from existing credentials and ZKPs We design and implement an anonymous credential system by creating zero-knowledge proofs (ZKPs) of existing credentials being issued today (as opposed to designing new credentials with privacy features built-in). The goal is to provide a ready-to-use library for multiple scenarios to showcase this approach and provide concrete benchmarks for all steps of the process. Our implementation includes two demo scenarios that illustrate the system end-to-end. Credentials are stored in a wallet, implemented as a browser extension. In the first scenario, an employment credential (in JWT format, that could be a Microsoft Entra or OpenID credential) is used to prove eligibility for employer provided mental health care benefits. Namely, the health care provider learns where the patient works, and nothing more, and the employer does not learn that the employee has accessed this benefit (which is highly personal). In the second scenario, using an mDL credential a user unlinkably proves they are at least 18 years old to create a new social media account, without revealing any other information from the mDL.

Technical Overview We observe that in each proof, almost all of the work is the same (the circuit cost is dominated by the signature verification proof). Thus, generating a single monolithic proof that repeats all work every time we want to present the credential is wasteful. Our design moves this common work to a pre-processing phase that the prover does once, after obtaining the credential. Since these credentials are long-lived, this happens infrequently. This offline work is relatively slow (tens of seconds), but subsequent proofs can be generated much faster (tens of ms), comparable to generating a BBS presentation proof. Proofs are about 1KB in size, often shorter than the input credential. (The size and time is dominated by a range proof to show the credential is not expired; that would also be required with BBS or other conceivable approaches).

Given this high-level design, we leverage mostly well-understood techniques to realize it. For the offline phase, we use the Groth16 [Gro16] proof system to create a proof of signed credential data that decodes and parses to a list of attributes that are then made outputs of the circuit. These outputs can be provided to the verifier in the form of a Pedersen vector commitment (as observed in LegoSNARK [CFQ19]), so they can be disclosed or kept hidden, as required by the scenario. The Groth16 proof and the commitment can be re-randomized, in a way that makes them unlinkable, so they can be computed once, and used repeatedly (as analyzed in [RWGM23]). Finally, we use a small Σ -proof to prove knowledge of the committed values, and a range proof [BFGW20] to show that the credential is not expired (or that age > 18). Since the final step is a proof about committed values, it is reminiscent of traditional anonymous credential systems [CV02, PZ23, LKWL25, CPY⁺, CPZ20], in particular it is very efficient. It’s also much easier to change the statement to be proven given a committed input, and we can easily interface with other sub-provers (such as range proofs, or other advanced predicates) in a modular way.

Some additional advantages follow from the fact that the performance of the offline SNARK prover step is not critical. We do not need special techniques to prove knowledge of the credential and can use one SNARK for multiple credential types. We can also afford to express the circuit in a more readable, but less efficient way (we use the Circom [ide] high-level language). The circuit may be common to all issuers that use the same signing algorithm, since it must only parse out the attributes. One limitation is the need for a trusted set of parameters; it’s important for verifiers to believe these were created honestly, requiring trust assumptions or distributed generation protocols. Fortunately, these have been well studied and deployed in the context of blockchains (see [NRBB24] for a recent survey).

Our implementation is a Rust library, together with Circom code expressing the circuits required for the offline phase. All code is available at <https://github.com/microsoft/crescent-credentials>. The main dependency is the arkworks library [ac22], it provides a Groth16 implementation and the primitives we need to implement the range proof and Σ -proof. In terms of performance, for a 2KB-long JWT signed with RSA-2048 and SHA-256, the offline phase (witness generation and proof generation done once per credential during their setup) takes 27s, showing a credential takes 22ms and verification takes 15ms. The proof is 987 bytes and the circuit has 1.5M constraints.⁴

To conclude, our work provides software demonstrating that strong privacy with existing credentials is practical. This will enable further experimentation, prototyping and research, a necessary step towards deployment of this technology.

2 Preliminaries

2.1 Overview of the Groth16 proof system

In this section we briefly review the [Gro16] proof system, in sufficient detail to explain how we use it to construct efficient credentials.

R1CS The circuits used in our construction will be expressed using a high-level language (Circom [ide]), which then compiles them to a rank-1 constraint system (R1CS) instance. The extended witness (often simply called the witness) is the input and output to the circuit, along with all intermediate wire values. An R1CS instance is a set of quadratic constraints on the witness values, such that a satisfying witness contains a valid (input, output) pair for the circuit. The circuit is an arithmetic circuit defined over a field $\mathbb{F}$, and all witness values, circuit inputs and outputs are elements of $\mathbb{F}$. Looking ahead, $\mathbb{F}$ will be the scalar field of a pairing-friendly curve, such as BN254 or BLS12-381.

More concretely, let the R1CS system have n variables, out of which p are public and we specify m constraints on them. The witness is defined as a vector $w \in \mathbb{F}^n$. In most codebases, the convention is to set $w_0 = 1$, to rule out the *trivial* solution of all zeros, and the next p entries are the public input. The constraint system is specified by three matrices $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ of size $n \times m$. A witness w satisfies the constraint system if $(w \cdot \mathbf{A}) \circ (w \cdot \mathbf{B}) = w \cdot \mathbf{C}$, where $\cdot$ denotes standard matrix/vector multiplication and $\circ$ denotes the Hadamard (element-wise) product.

QAPs A quadratic arithmetic program (QAP) is another way to represent an arithmetic circuit [GGPR13], and is the representation used by the Groth16 proof system. Therefore, the R1CS instance is first translated into a QAP. To interpret the R1CS constraints as a QAP:

- View the i -th row of each of the matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ as the evaluations of polynomials $A_i(X)$, $B_i(X)$, $C_i(X)$ on a fixed domain $\Omega = \{x_1, \dots, x_m\} \subset \mathbb{F}$ of size m , then compute A_i , B_i and C_i with polynomial interpolation. In practice Ω is chosen to be the roots of unity in order to use fast polynomial interpolation algorithms.
- Define polynomials $A(X) = \sum_{i=1}^m w_i A_i(X)$, and similarly for $B(X)$ and $C(X)$. Note that if $A(x_i)B(x_i) = C(x_i)$, then the i -th constraint of the R1CS instance is satisfied.

⁴Measured on a laptop with a 4-core Intel Core i7-10610U CPU @ 1.80GHz.

- From the above observation, it suffices to prove that the polynomial $A(X)B(X) - C(X)$ evaluates to 0 at all points in Ω . Let $Z_\Omega(X)$ be the vanishing polynomial on Ω : $Z_\Omega(X) = \prod_{i=1}^m (X - x_i)$. It is now sufficient to show that $A(X)B(X) - C(X) = Z_\Omega(X) \cdot H(X)$ for some “low-degree” polynomial $H(X)$.

2.1.1 The Groth16 Proof System

The proof system uses an elliptic curve with a pairing which we denote $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. Point G_1 is a generator of group $\mathbb{G}_1$ and G_2 generates $\mathbb{G}_2$. The groups have order r and the scalar field is $\mathbb{F}_r$. Below we use the notation $(a_1, \dots, a_n) \cdot G$ as shorthand for $(a_1 G, \dots, a_n G)$.

We describe a variant of Groth’s proof system that includes a simplification from [KMSV21] that preserves security, and allows proofs to be re-randomized [BKS21]. This re-randomization technique (also used in zk-creds [RWGM23]) is very efficient, and allows us to quickly generate presentation proofs of a credential.

Setup This function must be run once, and can be re-used for circuits with R1CS instances having m constraints or fewer. Sample random $\tau, \alpha, \beta \leftarrow \mathbb{F}_r$ and output the public parameters:

$$\begin{aligned} &(1, \tau, \tau^2, \dots, \tau^{2m-2}) \cdot G_1 \\ &(1, \tau, \tau^2, \dots, \tau^{m-1}) \cdot G_2 \\ &\alpha \cdot (1, \tau, \tau^2, \dots, \tau^{m-1}) \cdot G_1 \\ &\beta \cdot (1, \tau, \tau^2, \dots, \tau^{m-1}) \cdot G_1 \\ &\beta \cdot G_2 \end{aligned}$$

Circuit-Specific Setup. Once Setup has been run, a circuit-specific setup step is required for each circuit that will be used in proof generation and verification. Sample random $\delta \leftarrow \mathbb{F}_r$. Define polynomials L_i for $i \in [m]$ as:

$$L_i(X) = \beta \cdot A_i(X) + \alpha \cdot B_i(X) + C_i(X).$$

Next compute the prover key (required by proof generation), and the verifier key (required for proof verification). Note that the verifier key is much shorter than the prover key (recall that p is the number of public outputs).⁵

Prover Key:

$$\begin{aligned} &(\alpha, \beta, \delta) \cdot G_1 \\ &(1, \tau, \tau^2, \dots, \tau^{m-1}) \cdot G_1 \\ &\delta^{-1} \cdot (L_{p+1}(\tau), \dots, L_{m-1}(\tau)) \cdot G_1 \\ &\delta^{-1} \cdot (1, \tau, \tau^2, \dots, \tau^{m-2}) \cdot Z_\Omega(\tau) \cdot G_1 \\ &(\beta, \delta) \cdot G_2 \\ &(1, \tau, \tau^2, \dots, \tau^{m-1}) \cdot G_2 \end{aligned}$$

Verifier Key

$$\begin{aligned} &\alpha \cdot G_1 \\ &(L_0(\tau), L_1(\tau), \dots, L_p(\tau)) \cdot G_1 \\ &(\beta, \delta) \cdot G_2 \end{aligned}$$

The prover key and verifier key are public outputs.

Proof Generation Let $w = (1, w_1, \dots, w_m)$ be a witness that satisfies the R1CS instance specified by the matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C})$. Compute $A(X), B(X), C(X)$, and $H(X)$ as specified in Section 2.1 and $L(X) = \sum_{i=p}^{m-1} w_i \cdot L_i(X)$. Now, sample random $r, s \leftarrow \mathbb{F}_r$ and compute the proof:

$$\begin{aligned} A &= (\alpha + r \cdot \delta + A(\tau)) \cdot G_1 \\ B &= (\beta + s \cdot \delta + B(\tau)) \cdot G_2 \\ C &= (\delta^{-1}(L(\tau) + H(\tau) \cdot Z_\omega(\tau)) + s \cdot (\alpha + r \cdot \delta + A(\tau)) + r \cdot (\beta + s \cdot \delta + B(\tau)) - r \cdot s \cdot \delta) \cdot G_1 \end{aligned}$$

We use the notation (A, B, C) for both the proof, for and the polynomials in $\mathbb{F}_r[X]$, but it will be clear from the context which we are referring to.

⁵The terminology *prover key* and *verifier key* are common in the SNARK literature, but note that these values can be viewed as circuit-specific public parameters since they are public and shared by all provers and verifiers for a circuit.

Verification Given a proof (A, B, C) and public input $(w_0 = 1, w_1, \dots, w_p)$, the verifier checks that:

$$e(A, B) = e(\alpha \cdot G_1, \beta \cdot G_2) \cdot e(\bar{L}, G_2) \cdot e(C, \delta \cdot G_2), \quad (1)$$

where $\bar{L} = \sum_{i=0}^p w_i \cdot (L_i(\tau) \cdot G_1)$.

3 Anonymous Credentials from Existing Credentials

The Crescent construction is a type of anonymous credential system. In this section we describe anonymous credentials and the Crescent construction.

3.1 Overview of the Crescent Credential System

Existing Credentials By *existing credentials*, we are broadly referring to the class of digital credentials that are a list of attributes, signed with a public key signature. The Crescent implementation currently supports JWT and mDL credentials but X.509 certificates and machine readable passports are also in this class (see [RWGM23]). In Appendix A we give an example JWT for reference.

Parties An anonymous credential system has three types of parties.

Issuer: The Issuer (or *identity provider*) issues existing credentials to Users with a process that is out-of-scope for Crescent. The Issuer does not need to accommodate Crescent, nor even be aware of it.

User: The User (or *credential holder*) has a credential from the issuer that they will show using Crescent to one or more verifiers. The user generates proofs to send to verifiers, that are referred to as *show proofs* or *presentation proofs*.

Verifier: The Verifier (or *relying party*) receives proofs from one or more users, and if the proof is valid, provides access to a resource or service.

Protocols We informally describe the functionality as the six algorithms below along with some high-level comments describing Crescent. The existing credential system is assumed to have its own protocols, that are defined and run outside of Crescent; we denote them by (eSetup, eKeyGen, elssue, eVerify). Below we focus on describing how these interact with Crescent. For simplicity we assume a single existing credential type, and a fixed set of predicates that will be used when presenting the credential.

Setup After eSetup for the existing credential scheme has been run, the Setup algorithm for the anonymous credential system is run. Crescent requires that the Groth16 public parameters are generated, and the circuit-specific setup is run. **The circuit depends on the schema of the credentials (when it is flexible), and the cryptographic algorithms used by the issuer.**

KeyGen, Issue The key generation and issuance protocols are defined by the existing credential scheme. The Issuer runs eKeyGen, makes their public key available to all parties, and can then issue credentials to users with elssue.

Prepare The User runs Prepare once per issued credential. Prepare can be run anytime after Issue and before the first call to Show. Prepare creates *user state*, preprocessed information that is later used to Show the credential. The input to Prepare is a credential, the Issuer’s public key, the public and circuit-specific parameters, the prover and verifier keys. In Crescent Prepare creates a Groth16 proof, and parses out attributes from the credential. We note that the user state does not include the large prover key from Groth16 circuit-specific setup.

Show To create a show proof for a credential, Show needs the *user state* created by Prepare, and the predicate requested by the verifier. Example predicates are “age > 18” or “domain(email_address) = example.com”. Certain validation checks from the existing credential may be proven during Show rather than in the circuit – a common example is the expiry check; it cannot be done in the circuit since it

depends on the current time. The Crescent **Show** proof is a re-randomized instance of the Groth16 proof, and a small number of Σ -proofs about the committed attribute data. **Show** proofs can therefore be computed in a modular way, depending on the predicates requested by the verifier.

Verify The input to verification is the show proof, the issuer’s public key and any additional data required for predicates. If the proof is valid the **Verifier** accepts the predicate(s) as being satisfied by the credential (and any revealed data as authentic).

3.2 Detailed Description of Crescent

Circuits The circuit required for the proof generation step of **Prepare** must basically run the **Verify** algorithm of the existing credential scheme, as a normal verifier would. For JWTs this means:

1. Splitting the token into the header, payload and signature parts,
2. Base64-decoding these three values
3. Verifying the signature:
 - (a) hashing the payload and header to compute the digest, and then
 - (b) running the signature verification function (RSA or ECDSA) with the digest and issuer public key as inputs.
4. Parsing the JSON payload data to locate the attributes of interest, optionally applying a function to each attribute, then outputting them. The function could apply a collision-resistant hash to long attributes, encoding strings as $\mathbb{F}_r$ elements, or create derived attributes from existing ones (e.g., the email attribute could be divided into two parts: username and domain, so that the domain only can be revealed).

This circuit is non-trivial and the cost in R1CS constraints is high. For the example JWT given in Appendix A, which is about 2KB in length and signed with the RS256 algorithm (RSA PKCS#1v1.5 with SHA-256), the circuit has 1 583 999 constraints. This breaks down roughly as 1M for SHA-256 hashing, 250k for RSA verification, and 250k for everything else.

For mDL credentials the circuit has a similar high-level structure, but the attribute data is encoded with CBOR rather than JSON, and randomized hashes of the attributes are in the payload, rather than the values directly. The example mDL credential in Crescent is signed with ES256 (ECDSA on curve P256 with SHA-256), the signed data is 1152 bytes and the circuit has 2 635 013 constraints. **Here the dominant cost is ECDSA verification ($\approx 2M$ constraints), which requires two non-native scalar multiplications (i.e., the field operations required for scalar multiplication on the P256 curve are emulated in an arithmetic circuit on the BN254 curve used by our Groth16 implementation).**

3.3 Generating Show Proofs

Here we describe how Crescent uses one (expensive to generate) Groth16 proof for an unlimited number of efficient credential **Show** proofs, using an idea from [RWGM23].

The circuit outputs the parsed and formatted attribute values. The **Show** proof assigns to each attribute value one of the following types (chosen by the User, but depending on what must be proven to the verifier).

Hidden Nothing is proven about the attribute during **Show** (though something may have been proven in the circuit). Let $\mathcal{H}$ be the set of attribute indices having type Hidden.

Committed **The attribute is provided to the verifier in a separate commitment** (to only this attribute), so that a sub-prover may be run on this attribute. **This is a Pedersen commitment, so it is friendly to efficient Σ -proofs.** Let $\mathcal{C}$ be the set of attribute indices having type Committed.

Revealed The attribute is revealed to the verifier. Let $\mathcal{R}$ be the set of attribute indices having type Revealed.

Show proceeds as follows.

1. Let (A, B, C) be the Groth16 proof generated as described in Section 2.1 during Prepare and stored in the user state. Let the circuit public output values be $(\text{att}_1, \dots, \text{att}_p)$.
2. Re-randomize the Groth16 proof. Sample $\rho, \omega, z \leftarrow \mathbb{F}_r$ and compute

$$\begin{aligned}\pi_A &\leftarrow \rho^{-1} \cdot A \\ \pi_B &\leftarrow \rho \cdot B + \rho \cdot \omega \cdot (\delta \cdot G_2) \\ \pi_C &\leftarrow C + \omega \cdot A - z \cdot G_1. \\ \pi_{\bar{L}} &\leftarrow \bar{L} + z \cdot (\delta \cdot G_1)\end{aligned}$$

It can be checked that $(\pi_A, \pi_B, \pi_C, \pi_{\bar{L}})$ satisfy the Groth16 verification equation (Eq. (1)).

3. For each attribute att_i , if $i \in \mathcal{C}$, sample a fresh random value $z_i \leftarrow \mathbb{F}_r$ and compute a Pedersen commitment $\text{com}_i = \text{att}_i \cdot (L_i(\tau)G_1) + z_i \cdot (\delta G_1)$. The prover then updates $\pi'_C \leftarrow \pi_C - \sum_{i \in \mathcal{C}} z_i \cdot G_1$.
4. Generate a proof of knowledge of representation of $\pi_{\bar{L}}$ in the $(\{L_i(\tau) \cdot G_1\}_{i \in [p]}, \delta \cdot G_1)$ basis, where the scalars $\{\text{att}_i\}_{i \in \mathcal{R}}$ are revealed in the clear. The same proof also proves knowledge of the commitment openings, and in particular that com_i is a commitment to att_i . Denote this proof π_Σ , a proof of the relation

$$\left\{ (\{\text{att}_i\}_{i \in \mathcal{H}}, \{\text{att}_i, z_i\}_{i \in \mathcal{C}}) : \pi_{\bar{L}} = z(\delta G_1) + \sum_{i=1}^p \text{att}_i (L_i(\tau)G_1) \wedge \text{com}_i = \text{att}_i \cdot (L_i(\tau)G_1) + z_i \cdot (\delta G_1) \right\}$$

Instantiating (non-interactive) proofs of knowledge for this type of relation is well-known (see, e.g., [BS23, §20.2]).

5. Output $(\pi_A, \pi_B, \pi_C, \pi_{\bar{L}}, \{\text{att}_i\}_{i \in \mathcal{R}}, \{\text{com}_i\}_{i \in \mathcal{C}}, \pi_\Sigma)$

Verify Aggregate the commitments into π_L , by computing $\pi'_L \leftarrow \pi_{\bar{L}} + \sum_{i \in \mathcal{C}} \text{com}_i$. Then verify $(\pi_A, \pi_B, \pi_C, \pi'_L)$ with Eq. (1). Finally, verify π_Σ . If all checks pass, accept the Show proof as valid.

Sub-provers for Show Using the hidden and revealed attribute types we can implement *selective disclosure* where the prover reveals a subset of the attributes to the verifier. With the committed type and an additional *sub-prover* we can implement more advanced predicates. The example implemented in Crescent is a module for range proofs, that we use to prove that JWTs and mDLs are not expired, and that the mDL birth date encodes an age greater than a given constant.

While there is some overhead to creating fresh commitments for sub-provers and proving they are consistent with the Groth16 vector commitment, we decided that the modularity this implementation provides is worth the cost. The alternative is to run all sub-provers in a monolithic proof. The consistency proof is an equality of discrete logarithms proof (DLEQ proof) that has a well-known and simple Σ -proof. Another natural extension enabled by committed attributes is the ability to link credentials together, for example, it is easy to prove that the name field of an employment JWT matches the name of the mDL. Simply create Show proofs for both credentials and make the name field committed. Then use a DLEQ proof to show the committed names are equal. This feature was also used in [RWGM23].

The DLEQ proof is also a natural place to bind context, session or nonce data to the proof, e.g., to ensure a proof is not being replayed, a verifier may send a nonce to a prover. Since we use the Fiat-Shamir [FS87] transform to make this proof non-interactive, we can efficiently bind data to the proof by hashing it when generating the challenge. Proofs that authenticate data in this way are sometimes called *signatures of knowledge* [CS97, CL06, GM17], and Crescent can also be used to create signatures, without the need for signers to manage a long-term signing key (which has proven difficult in practice). For example, a developer could (pseudonymously) sign code or commit messages by proving knowledge of an existing credential.⁶

⁶Care must be taken here not to use a credential that is frequently presented to many verifiers, since a malicious verifier could use the credential to generate a signature. We offer two potential mitigations: (i) use a set of credentials or (ii) during credential issuance, if the user is allowed to provide a nonce value that is included in the token, it can be made the hash of a secret value, and the circuit can check that the prover knows this value.

We have also prototyped a method to efficiently form committed attributes with a ZK-friendly hash function to more efficiently use (zk-)SNARKs as sub-provers, however decided to hold this (rather complex) feature back, until a compelling scenario demands it.

Circuits and multiple issuers Since the per-circuit parameters are large, and must be generated by a trusted party or with a distributed protocol, we would like to have as few parameter sets possible. With our current implementation, when issuers share the same schema and the same cryptographic signing algorithm, the same circuit may be used. More precisely, if the set of attributes that the circuit must output are present in all tokens, then same circuit will work (since it will ignore other attributes). We expect the mDL issuers to be consistent, at least at the country level (e.g., AAMVA has a profile [oMVA24] of the ISO mDL standard for the United States), but more investigation is required. We note that it is possible to create a more general circuit, that takes a list of attribute names as input, to allow sharing between issuers with different schemas but using the same signature algorithm.

3.4 Device Bound Credentials

Credentials are often bound to a device, with the goal of preventing use of the credential without possession of the device. To implement this feature, a signing key pair is generated by the device, and the private key is stored in hardware. This hardware provides an HSM-like functionality, where private keys are never exported, but can be used via a fixed set of APIs. The public key is sent to the issuer, and is encoded as one of the attributes in the credential. When the credential is presented, the verifier sends a random challenge to the user, who asks the device to sign it, and the signature is included as part of the credential presentation.

To maintain unlinkability with device binding, the device public key must remain hidden from the verifier. The approach we take in Crescent is to define a sub-prover, that takes as input a commitment to the public key, and proves knowledge of a signature that verifies with the committed public key.

We note some assumptions before describing the protocol.

- The devices are assumed to be standard secure elements, such as those found on PCs and mobile phones. As such, they support a limited number of very common signature algorithms, such as RSA and ECDSA. In our Crescent implementation we currently support ECDSA on the NIST-P256 curve with SHA-256.
- We assume the device is not maliciously trying to break unlinkability. In particular the device generates signatures honestly, and does not try to encode information about the user that would break unlinkability [CP93]. This allows us to make part of the signature public, reducing the cost of generating the proof.
- If the device uses a derandomized ECDSA implementation, such that the signature on the same message produces the same signature, we assume that the message to be signed includes randomness from the credential holder or the current time, to ensure that each presentation proof has a fresh signature.

3.4.1 Linking proofs

The device binding sub-prover takes as input a commitment to the public key; this commitment is in the group $\mathbb{G}_1$ of order r defined by the Groth16 parameters. Our sub-prover uses a different group, denoted $\mathbb{G}_p$, chosen to make the ECDSA proof more efficient (using techniques described in [WOS⁺25]). We now describe how these proofs are linked together. This technique can be used more generally, to “export” data from the credential to other sub-provers as well.

Device public key We denote the public key by $Q = (q_0, q_1)$ where Q is the x -coordinate of the ECDSA key, and $Q = q_0 + 2^{128}q_1$. The key is split into two 128-bit parts since it’s larger than r (the group order of $\mathbb{G}_1$). The Groth16 circuit locates the device public key in the credential, converts it to an integer, and makes the two 128-bit pieces outputs of the circuit.

Linker input During a show proof, the attribute values (q_0, q_1) are committed attributes, and the linker is given commitments and openings: $(\text{com}_0 = G^{q_0} H^{r_0}, q_0, r_0)$ and $(\text{com}_1 = G^{q_1} H^{r_1}, q_1, r_1)$.

Linking proof The proof requires two hash functions H and H_p . The hash function H_p is chosen to be efficient as a circuit in $\mathbb{Z}_p$ (we use Poseidon), while for H we use SHA-256. For simplicity we assume that $r < p$ in our presentation (and this is currently the case in our implementation). The prover performs the following steps:

1. Sample random values (z, r_z) from $\mathbb{Z}_r$ and create the commitment $\text{com}_z = G^z H^{r_z}$.
 2. Compute the digest $h_Q = H_p(q_0, q_1, z)$.
 3. Create a random linear combination of the commitments as follows:
 - (a) Compute a challenge: $(e_1, e_2) = H(\text{com}_0, \text{com}_1, \text{com}_z, h_Q)$.
 - (b) Compute $v = q_0 + e_1 q_1 + e_2 z \pmod{r}$ and $\text{com}_v = \text{com}_0 \cdot \text{com}_1^{e_1} \cdot \text{com}_z^{e_2}$.
 4. Create a Σ -proof, denoted π_1 , that $\{(a, z, r_z) : \text{com}_v / G^v = H^a \wedge \text{com}_z = G^z H^{r_z}\}$. (The proof does not need to prove anything about a , but the honest prover will compute $a = r_0 + r_1 e_1 + r_z e_2$.)
 5. Create a SNARK proof, denoted π_2 with public inputs (h_Q, v, e_1, e_2, T, U) , and private inputs (q_0, q_1, z, s) . (The values (T, U, s) are for signature verification, defined below). Proof π_2 proves:
 - (a) $h_Q = H_p(q_0, q_1, z)$
 - (b) $v = q_0 + q_1 e_1 + z e_2 \pmod{r}$
 - (c) $T^s U = Q$ where $Q = q_0 + q_1 2^{128}$
- The third step is for ECDSA signature verification, we describe the details below.
6. Output $(\text{com}_0, \text{com}_1, \text{com}_z, h_Q, m, \pi_1, \pi_2)$

Our linking proof is similar to the protocol Π_{dlhash} given in [OKMZ24], but is optimized to reduce the amount of hashing and mod r arithmetic in the SNARK circuit.

Linking proof verification

1. Recompute (e_1, e_2) and com_v as above.
2. Use $v, \text{com}_v, \text{com}_z$ as public values and ensure π_1 is valid.
3. Use (h_Q, v, e_1, e_2, T, U) as public inputs and ensure π_2 is valid.

ECDSA Signature Proof The SNARK used for π_2 in the linking proof is chosen to be convenient for ECDSA proofs of possession. We use the Spartan [Set20] proof system, instantiated over the T-256 curve [FLM22], as in [WOS⁺25]. The group order of this curve, denoted p , is the P-256 prime, so that all curve group operations have efficient arithmetic circuits mod p . For example, a scalar multiplication requires approximately 2700 R1CS constraints.

Let (r, s) denote an ECDSA signature on message M (we assume M has already been hashed and converted to an integer modulo the group order). The verification equation is

$$r = f(Q^{r/s} G^{M/s})$$

where f takes a point on the curve and returns the x -coordinate. Equivalently, given (R, s) where $R = f^{-1}(r)$, the verifier can compute $r = f(R)$ and check that $R = Q^{r/s} G^{M/s}$. This can be re-written as

$$T^s U = Q \quad \text{where} \quad T = R^{1/r} \quad \text{and} \quad U = G^{-M/r}$$

Since M is public, and R is a random value, the prover can reveal R and the verifier can recompute (T, U) themselves.⁷ This gives us the final step of the SNARK π_2 , using the additional public inputs (T, U) and private input s , the circuit checks that $T^s U = Q$, using one scalar multiplication and one point addition.

⁷This modified signing equation was also used here: <https://github.com/personaelabs/spartan-ecdsa>

Unlinkability Device binding adds linking and signature proofs to our **Show** proof, and they must preserve unlinkability of the main proof. Below we show that the linking proof is a statistical zero-knowledge argument, and the signature proof computed with Spartan has perfect zero-knowledge (we use the “SpartanNIZK” variant of Spartan [Set20]).

Unforgeability Let Q be the x -coordinate of the device bound ECDSA credential. From the knowledge soundness of the Groth16 proof system, correctness of the Groth16 circuit, and the Σ -protocols in Section 3.3, it must be the case that com_0 and com_1 are commitments to q_0 and q_1 in $\mathbb{F}_r$, such that $Q = \phi_p(q_0) + 2^{128} \cdot \phi_p(q_1) \pmod p$.⁸

Now we argue that the linking proof proves knowledge of a signature σ on a publicly known message M under an ECDSA verification key with x -coordinate Q defined as above. Observe that if we can build a sound NIZK for the following relation:

$$\{q_0, q_1, r_0, r_1, \sigma \mid \text{com}_0 = \text{Com}(q_0; r_0) \wedge \text{com}_1 = \text{Com}(q_1; r_1) \wedge Q = \phi_p(q_0) + 2^{128} \cdot \phi_p(q_1) \wedge \text{ECDSA.Verify}(Q, \sigma, M) = 1\}, \quad (2)$$

then we are done. As an intermediate step consider the following interactive protocol.

1. The prover samples random values (z, r_z) from $\mathbb{F}_r$ and creates a commitment $\text{com}_z = \text{Com}(z; r_z)$.
2. The prover computes $\text{com}_p = \text{Com}_p((q_0, q_1); z)$, where Com_p is a second commitment scheme.
3. The prover sends $\text{com}_0, \text{com}_1, \text{com}_z$ and com_p to the verifier.
4. The verifier samples a random challenge $(e_1, e_2) \leftarrow \mathbb{F}_r^2$ and sends it to the prover.
5. The prover responds with $(q_0, q_1, z, r_1, r_z, q'_0, q'_1, z')$.
6. The verifier accepts if
 - $\text{com}_0 = \text{Com}(q_0; r_0)$
 - $\text{com}_1 = \text{Com}(q_1; r_1)$
 - $\text{com}_z = \text{Com}(z; r_z)$
 - $\text{com}_p = \text{Com}_p((q'_0, q'_1); z')$
 - $q_0 + e_1 \cdot q_1 + e_2 \cdot z = q'_0 + e_1 \cdot q'_1 + e_2 \cdot z' \pmod r$

Lemma 1. *If Com and Com_p are computationally binding commitment schemes, then for all PPT provers P^* , we have that*

$$\Pr[(q_0, q_1, z) \neq (q'_0, q'_1, z') \mid \langle P^*, V \rangle \rightarrow 1] \leq \text{negl}(\lambda)$$

Proof. From the computational binding property of the commitment schemes, P^* can open the commitments to at most one possible value, except with negligible probability. Next observe that the last check holds, for any pair of triples $(q_0, q_1, z) \neq (q'_0, q'_1, z')$, if the non-zero polynomial $F(X_1, X_2) = X_2(z - z') + X_1(q_1 - q'_1) + (q_0 - q'_0)$ is zero at (e_1, e_2) . Since the coefficients of F are fixed before (e_1, e_2) is randomly chosen, the probability is $1/r$ (by the Schwartz-Zippel-Lipton-DeMilo lemma). Thus, the verifier will reject, except with probability $1/r$ if $(q_0, q_1, z) \neq (q'_0, q'_1, z')$. $\square$

Intuitively, the above protocol shows that even though the witness (q_0, q_1) is committed to in two *different* ways using Com and Com_p , if we have an additional check as outlined above, then we are guaranteed that the underlying witness in the two commitment schemes is the same. We now present an interactive protocol for the full relation Eq. (2) which can be made non-interactive using the Fiat-Shamir heuristic [FS87]. It aims to achieve the same goal as the protocol above, while preserving the privacy of the witness (q_0, q_1) , by *efficiently* replacing the verifier of the last step with a NIZK.

1. The prover samples random values (z, r_z) from $\mathbb{F}_r$ and creates a commitment $\text{com}_z = \text{Com}(z; r_z)$.

⁸ $\phi_p : \mathbb{F}_r \rightarrow \mathbb{F}_p$ is the natural injective map. Since $r < p$, this map is well defined on all of $\mathbb{F}_r$.

2. The prover computes a commitment $\text{com}_p = \text{Com}_p((q_0, q_1); z)$, where Com_p is a commitment scheme that can be computed efficiently using an $\mathbb{F}_p$ -arithmetic circuit.
3. The prover sends $\text{com}_0, \text{com}_1, \text{com}_z$ and com_p to the verifier.
4. The verifier samples a random challenge $(e_1, e_2) \leftarrow \mathbb{F}_r^2$ and sends it to the prover.
5. The prover computes $v = q_0 + e_1 \cdot q_1 + e_2 \cdot z \pmod{r}$, and sends v to the verifier along with NIZKs for the following relations:

$$\Pi_r := \{q_0, q_1, z, r_1, r_1, r_z \mid \text{com}_0 = \text{Com}(q_0; r_0) \wedge \text{com}_1 = \text{Com}(q_1; r_1) \wedge \text{com}_z = \text{Com}(z; r_z) \wedge v = q_0 + e_1 \cdot q_1 + e_2 \cdot z \pmod{r}\}$$

$$\Pi_p := \{q_0, q_1, z, \sigma \mid \text{com}_p = \text{Com}_p((q_0, q_1); z) \wedge v = q_0 + e_1 \cdot q_1 + e_2 \cdot z \pmod{r} \wedge Q = \phi_p(q_0) + 2^{128} \cdot \phi_p(q_1) \wedge \text{ECDSA.Verify}(Q, \sigma, M) = 1\}$$

where Π_r , and Π_p are instantiated using proof systems that have an efficient prover for $\mathbb{F}_r$, and Π_p -arithmetic circuits, respectively.

6. The verifier accepts if Π_r and Π_p pass verification.

Theorem 1. *If Com is a perfectly hiding and computationally binding commitment scheme, Com_p is a statistically hiding and computationally binding commitment scheme, and Π_r and Π_p are NIZKs that satisfy knowledge soundness with perfect zero-knowledge, then the protocol above is an honest verifier argument of knowledge with statistical zero-knowledge for the relation in Eq. (2).*

Proof. Completeness follows from completeness of the underlying NIZKs. Knowledge soundness follows by running the extractor for Π_r and Π_p in combination with Lemma 1 which guarantees that the witness (q_0, q_1, z) extracted from Π_r and Π_p is identical.

Zero Knowledge. The simulator is given the statement $(\text{com}_0, \text{com}_1, M)$. It samples $\tilde{v} \leftarrow \mathbb{F}_r$ and sets com_z and com_p to be commitments to values $(\tilde{q}_0, \tilde{q}_1, \tilde{z})$ that are sampled from the appropriate distribution. The value $\tilde{z} \leftarrow \mathbb{F}_r$ but $\tilde{q}_0$ and $\tilde{q}_1$ are computed by first sampling an ECDSA public key and then decomposing the x -coordinate. It now runs the simulators for Π_r with statement $(\text{com}_0, \text{com}_1, \text{com}_z, \tilde{v})$ and Π_p with statement $(\text{com}_p, \tilde{v}, M)$ to simulate the NIZKs. Finally, it sets the challenge $\tilde{e}_1 \leftarrow \mathbb{F}_r$ and $\tilde{e}_2 = (\tilde{v} - \tilde{q}_0 - \tilde{q}_1 \cdot e_1) / \tilde{z}$ and outputs $(\text{com}_z, \text{com}_p, \tilde{c}, \tilde{v}, \Pi_r, \Pi_p)$ as the simulated proof.

To show that the simulated transcript is computationally indistinguishable, we will go through a sequence of hybrids starting from the real execution of the protocol.

1. In the first hybrid, the transcript is computed using the witness by following the protocol faithfully.
2. Because Com is perfectly hiding, the statement proved by Π_r is always true. In this hybrid, we simulate the proof Π_r by running the corresponding simulator and this switch is perfectly indistinguishable as Π_r satisfies perfect zero-knowledge.
3. Because there always exists a signature on any string M in the ECDSA scheme, we are guaranteed the statement proved by Π_p is also always true. In this hybrid we simulate the proof Π_p by running the corresponding simulator and this switch is perfectly indistinguishable.
4. Next, we sample $v \leftarrow \mathbb{F}_r$, and $e_1 \leftarrow \mathbb{F}_r$ and set $e_2 \leftarrow (v - q_0 - q_1 \cdot e_1) / z$. This hybrid is distributed identically to the previous hybrid.
5. In the final hybrid we sample $\tilde{v} \leftarrow \mathbb{F}_r$ and set com_z and com_p to be commitments to $(\tilde{q}_0, \tilde{q}_1, \tilde{z})$ which are sampled as outlined above. Following the previous hybrid, we set $\tilde{e}_1 \leftarrow \mathbb{F}_r$ and $\tilde{e}_2 \leftarrow (\tilde{v} - \tilde{q}_0 - \tilde{q}_1 \cdot e_1) / \tilde{z}$. This switch is statistically indistinguishable from the hiding property of Com_p . We note that our instantiation of Com_p with Poseidon is statistically hiding since we commit to (q_0, q_1) as $\text{H}_p(q_0, q_1, z)$ and the length of $q_0 || q_1$ and z are the same, and each are as long as the output.

□

An optimization in our implementation is that the ECDSA signature verification equation is the modified one described above, and as part of the proof we reveal the signature value R . Since we assume the device computes signatures honestly, R is a uniformly random group element in $\mathbb{G}_p$. The zero-knowledge simulator can therefore simulate R with a random value.

3.4.2 Performance

The size of the circuit we must prove with Spartan has 4409 constraints; most of this is for the elliptic curve scalar multiplication (2700 constraints) and the emulated field arithmetic mod r (1200 constraints). On our test machine (details in Section 4), the prover time is about 300 ms, verifier time is about 160 ms and the proof size is 13.1 KB. The public parameters required by the prover and verifier are not currently stored, we simply re-derive them for every call to prove and verify, this costs 10 ms. The size of these parameters is 4.5 KB.

3.4.3 Optimizations and Extensions

Some additional optimizations are possible, which we may investigate and implement in the future.

Pre-computing SNARKs for ECDSA We describe an optimization that allows pre-computing the SNARK. Using the modified verification equation, we can randomize it as

$$T^s U D^y = Q D^y$$

where D is a fixed random group element and y is a fresh random scalar. Let B denote the group element in this equation (i.e., $B = T^s U D^y = Q D^y$). The prover can make B public. To prove knowledge of a signature, the prover must convince the verifier that (i) $B = T^s U D^y$ and (ii) $B = Q D^y$ hold. Condition (i) depends on the message and the signature, these are not known in advance, but fortunately this condition can be proven with a simple Σ -proof. Condition (ii) requires the (more expensive) SNARK, and it fortunately depends only on Q and y , and can therefore be computed in advance (along with the linking proof). Therefore in situations where the latency introduced by generating the SNARK proof is material, the SNARK can be precomputed, restoring the fast presentation times of Crescent without device binding.

Improved proofs of mixed statements By observing that the core of the problem is proving knowledge of b, C , such that $A^b = C$, we can use the techniques in [WQZD24] that extend the work of [OKMZ24] to the case when C is hidden, and provide some optimizations to reduce the cost of scalar multiplications in the circuit by moving parts of the computation outside the circuit, and also allowing a lower soundness security target.

Direct linking proofs Our approach to linking the Groth16 proof to the proof of device signature is generic in that it works with any SNARK to compute π_2 . This introduces some overhead in the circuit, namely we have some non-native arithmetic (mod r) and Poseidon hashing, which accounts for roughly 40% of the constraints. We note that it is possible to avoid this overhead in the circuit, with a linking proof that is specialized to Spartan. As part of Spartan the prover must commit to the private inputs, in our case these are (q_0, q_1) . With a linking proof that shows that openings (q_0, q_1) of $(\text{com}_0, \text{com}_1)$ are committed correctly in Spartan, we have assurance that the two proofs are correctly linked. Both are variants of Pedersen commitments, but in groups of different order. The Σ -proof given in [OKMZ24] is the most efficient option, and the requirement that q_i be bounded in size is already ensured by the Groth16 circuit. We expect the savings from the SNARK proof with fewer constraints to outweigh the the additional cost of this Σ -proof. Direct linking proofs along these lines should also exist for other group-based proof systems as well (such as Bulletproofs [BBB⁺18]).

Credential Private Keys and Rate Limiting The linking proof defined for device binding can also be used to add a private key to existing credentials, when the issuance protocol allows the user to submit an attribute (such as a nonce, as in zkLogin [BCJ⁺24]). For example, the user can set the nonce to be $y = H_p(x)$, where x is a random, secret value, H_p is the Poseidon hash function. Then when presenting the credential, the nonce attribute is committed, and linked to a SNARK proof of knowledge of x . This is weaker than device binding (since x is not stored in hardware), but still ensures that the user presenting the credential knows the private key every time the credential is used. We estimate the cost of generating and verifying the SNARK proof to be about 2/3 of the cost for device binding (approximately 2000 R1CS constraints), with similar proof sizes. Note that in this scenario Spartan does not need to be instantiated on the T-256 curve, and further performance gains are possible by using a faster curve.

The credential private key x can also be used to implement rate limiting, where a user is allowed to present their credential to a verifier up to k times. The user creates a public tag value for each presentation, $t = H_p(x, \text{context}, i)$ and while proving knowledge of x , also proves that t is computed correctly and that $i \in \{1, \dots, k\}$. The verifier accepts the presentation if the tag value has not been previously used. Since t is computed deterministically from x , each credential can only create k distinct tags.

3.5 Security

Intuitively the security properties we want from this system are the following.

Correctness ensures that for all valid credentials and sets of predicates, the **User** can generate a proof that will be accepted by the **Verifier**. This depends on the correctness of both the Groth16 and Σ -proofs, as well as the correctness of the circuit.

Unforgeability ensures that a malicious user cannot produce an accepting proof for a set of predicates, unless they have a valid credential satisfying those predicates. Again this reduces to soundness of the underlying proof systems that make up Crescent and correctness of the circuit. Additionally, it requires that the existing credentials be unforgeable.

Anonymity/Unlinkability ensures that proofs reveal only what is revealed by the predicates that are proven. This strong privacy ensures that users who do not reveal identifying information can remain anonymous. This implies unlinkability, since a verifier that receives two **Show** proofs cannot distinguish whether they were created by the same or different **Users** (provided the predicates and revealed data are the same). This property reduces to the zero-knowledge property of the underlying proof systems. As mentioned, the Groth16 system requires a circuit-specific setup to be generated with a ceremony, or by a trusted party. We can show that even with knowledge of the trapdoor (τ), it is not possible to link a **User** across different **Show** proofs.

We add the disclaimer that we have not yet done a formal analysis of Crescent as a whole, and the above properties are not proven, just what we expect will hold. The primitives we rely on are well analyzed. There are many existing analyses of Groth16 and variants of it [Gro16, BKS^V21, CFQ19, GM17, RWGM23], the Σ -proofs are also straightforward to analyze [BS23] and the range proof we use is built on well-understood polynomial commitments [BFGW20, KZG10, MBKM19].

Quantum attacks NIST recently standardized new cryptographic algorithms that are expected to resist attacks using quantum computers. The initial goal of the transition to these post-quantum algorithms is to reduce the risk of a “record-now decrypt-later” attack, where an attacker records ciphertext today, protected with current algorithms, and potentially decrypts it in the future if they obtain a sufficiently capable quantum computer. Signature and authentication algorithms are also being migrated in some limited settings (e.g., for code signing and software update pipelines), but generally with a lower priority for online authentication scenarios, as there are no retroactive attacks like with confidentiality.

Therefore, many standards used in the identity space (such as JWT, and also new standards like mDL) are not currently using post-quantum signature algorithms. The credentials are relatively short-lived and can be updated when these systems migrate to post-quantum cryptography.

The Groth16 proof system and the other proof techniques we use in Crescent have perfect and statistical zero-knowledge, which means that an attacker recording **Show** proofs today will not be able to learn additional

user data given a quantum computer (or even a hypothetical machine much more capable than a quantum computer). Thus the privacy and unlinkability guarantees Crescent provides can be considered post-quantum secure.

What an attacker can do with a quantum computer is break soundness of the proof system, meaning they can create **Show** proofs, without having a valid credential, that a verifier will accept. This is because the soundness (or unforgeability) guarantees of Crescent depend on computational assumptions that a quantum computer can break. However, breaking these assumptions also means that the **Issuer**’s signature algorithms are also broken by quantum attacks, and we have to update the existing credential standards as well as the software of all parties (at which point we could also update the proof system).

Finally, currently known zero-knowledge proofs with post-quantum zero-knowledge and soundness that apply to the credential scenarios we are considering are much newer and less well studied than Groth16 and Σ -proofs. These new constructions often rely on new assumptions (or new parameter sets, e.g., with lattices), and they come with the main performance drawback of having larger proof sizes (hundreds of kilobytes).

4 Benchmarks

We report some benchmarks showing the CPU time required for Crescent operations, as well as the sizes of Crescent proofs and parameters. The code we benchmark is available at <https://github.com/microsoft/crescent-credentials>. In the first JWT example (labeled “rs256” in the Table 1), we create a proof that discloses the domain name of the `email` attribute. In the next example (labeled “rs256-sd”), the circuit has support for selective disclosure. Eight attributes from the JWT are circuit outputs (the **Show** proof can reveal any subset of these), and two of them are strings too large to fit in a field element and are hashed first (with SHA-256). In the **Show** proof, four of eight attributes are disclosed. Finally, the third JWT example (labeled “rs256-db”), is the same as the selective disclosure example, but the credential is device bound and presentation includes a proof of the device signature. All three examples use a test token similar to the one shown in Appendix A, which is about 2KB in length and signed with the RS256 algorithm (RSA PKCS#1v1.5 with SHA-256). In the mDL example, we prove that the holder is older than 18 years old. The credential is signed with ECDSA on curve P256 with SHA-256, the signed data is 1152 bytes and the circuit has 2.6M constraints. We have not yet added support for device-bound mDL credentials, but expect the costs to increase by the same amount as in the rs256-db case.

Our implementation currently supports the BN254 elliptic curve (the curve used by Ethereum for smart contracts [BR17]). Our test machine is an Intel Xeon W-2133 CPU @ 3.60GHz (launched 2017), with 6 cores. The `arkworks` library and the Spartan-T256 implementation we depend on for cryptographic computations use multiple cores, primarily to accelerate multi-scalar multiplication operations.

The results are given in Table 1. Credentials that are not device bound have a similar performance profile and cost tradeoffs. The time for creating and verifying **Show** proofs is low, comparable to purpose-built anonymous credential systems like BBS [LKW25]. The size of proofs is also small – note that in the JWT case, the proof is about half the size of the token, so the required bandwidth is actually smaller when showing a token with a ZKP. In the mDL case, our proof is larger (but still comparable in size to using the mDL without a ZKP), since we require two range proofs (each is 636 bytes): one to prove the credential’s valid until date is in the future, and one for the age predicate. We have not yet investigated batching these range proofs to reduce costs. The user state and parameters required for **Show** and **Verify** are also small.

When credentials are device bound, the cost of **Show** and **Verify** and the proof size all increase significantly, but are still low enough to be practical in many scenarios. Some directions for reducing costs for device-bound credentials are given in Section 3.4.3.

The drawbacks are typical of applications using the Groth16 (or similar) proof system. The time for **Prepare**, where we generate the one-time Groth16 proof for a credential, is tens of seconds. We note that the Circom-generated WASM code for witness generation in the mDL case is highly inefficient, we suspect due to some large lookup tables present in the ECDSA circuit we are currently using. We expect that this will be in line with the JWT times (**Prepare** taking ≈ 35 s) after some optimization effort. The other drawback is the large size of the parameters required for the **Prepare** step (594 MB to 1.1 GB). Fortunately these are only required during **Prepare**, which is infrequent, and the parameters required during the more common operations are small.

	JWT			mDL
	rs256	rs256-sd	rs256-db	
Prepare	19.9 s	22.0 s	26.4	140 s
Witness gen.	7.5 s	7.6 s	8.0 s	129 s*
Groth16 prove	12.4 s	14.4 s	18.4 s	21 s
R1CS constraints	1.6 M	1.9 M	2 M	2.6 M
Show	29.2 ms	28.3 ms	315 ms	41.2 ms
Verify	11.7 ms	11.4 ms	184 ms	24.4 ms
User state	40 KB	41 KB	42 KB	39 KB
Show proof	1019 B	1083 B	15083 B	1727 B
Prepare params	593 MB	649 MB	661 MB	1.1 GB
Show params	56 KB	56 KB	56 KB	56 KB
Verifier params	40 KB	40 KB	40 KB	40 KB

Table 1: Benchmark results (March 2025). Timings are given in the top half of the table and sizes in the bottom half. Three RSA-signed JWT credentials and one mDL are used for benchmarking (details of these are given in the text).

Our range prover is an implementation of the proof described in [BFGW20]. For the 32-bit range we need, the size of the proof is 636 bytes, proofs take 9.8 ms to generate and 6.3 ms to verify. The size of the prover key is 17 KB and the verifier key is 640 bytes. As it is based on polynomial commitments, it also requires a trusted setup and a pairing-friendly group. In the context of our credential system, these could be generated at the same time as the Groth16 parameters. Our implementation currently uses the same EC implementation from arkworks (the BN254 curve).

5 Additional discussion of related work

In zk-creds [RWGM23], the existing credential issuer (e.g., the passport issuer) is oblivious to the zk-creds system and does not need to modify how they issue credentials. However, the zk-creds system adds a new functionality, a type of issuer, that maintains a list of credentials that have been issued, in the form of a membership list (encoded as a Merkle tree with a public root and idealized as a “*bulletin board instantiated as a transparency log, Byzantine system, or even a blockchain*”). A credential is added to the membership list, after a user presents *zk-supporting documentation* (which could take the form of a ZKP of a valid passport). A commitment to the zk-supporting documentation is also logged in the membership list. The system only uses the existing credential for vetting during the issuance phase, then “*we need only trust the issuer to add credentials (and their supporting documentation) to the list*”. Once this new credential is issued, presenting it is much faster (150ms) since it requires only proving knowledge of inclusion in a Merkle tree. As partial justification for this new design, the paper argues that the folklore approach of Cinderella and Crescent is insufficient (Section 7.1), with the two sentences: “*A proof over a certificate or indeed any existing credential is insufficient: it may not include the necessary data and that data cannot be hidden from the issuer. This is a challenge for, e.g., the random seed we use for cloning resistance. As a result, we must issue our own standalone credentials and can, at best, use existing certificates as supporting documentation to justify issuance.*”. We argue that working around this limitation (if needed), and avoiding new issuance infrastructure makes Crescent easier to understand, but more importantly easier to deploy. First, the need for extra data is only required for some features, and these can likely be obtained with another approach. (e.g., the random seed can be the hash of two existing credentials, issued by different issuers). Second, with some types of existing credentials (such as OpenID), the user can include a nonce which can encode arbitrary data hidden from the issuer, as in zkLogin, which they use to bind a signing key to an OpenID identity.

The **zkLogin**[BCJ⁺24] system also uses existing credentials, but to authenticate blockchain transactions. They focus on using JWTs issued for OpenID authentication. In zkLogin prover time was also considered to be prohibitively high for frequent authentication, and the paper addresses the issue by offloading proof generation to a trusted party with ample resources, and by binding a signing key to the OpenID token (by re-purposing the nonce field in the issuance protocol). The signing key is then used as a pseudonym.

Crescent aims to work independent of the blockchain application, with multiple existing credential types, and support full anonymity.

6 Acknowledgments

We thank Larry Joy for help with the implementation of the demos included with the Crescent library. Thanks also to Tianyi Liu, who contributed to some early prototyping work. Finally, we thank Srinath Setty, Weidong Cui and Sebastian Angel for helpful discussions on this topic.

References

- [ac22] arkworks contributors. **arkworks** zkSNARK ecosystem, 2022. <https://arkworks.rs>. Presented at RWC 2022.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- [BCJ⁺24] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindstrøm, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zklogin: Privacy-preserving blockchain authentication with existing credentials, 2024.
- [BFGW20] Dan Boneh, Ben Fisch, Ariel Gabizon, and Zac Williamson. A simple range proof from polynomial commitments, 2020.
- [BKSV21] Karim Bagheri, Markulf Kohlweiss, Janno Siim, and Mikhail Volkhov. Another look at extraction and randomization of Groth’s zk-SNARK. In *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part I 25*, pages 457–475. Springer, 2021.
- [BR17] Vitalik Buterin and Christian Reitwiessner. EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt.bn128. Technical report, February 2017.
- [BS23] Dan Boneh and Victor Shoup. A graduate course in applied cryptography, 2023. Available online <https://toc.cryptobook.us/>.
- [CFQ19] Matteo Campanelli, Dario Fiore, and Anaïs Querol. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2075–2092. ACM Press, November 2019.
- [CL06] Melissa Chase and Anna Lysyanskaya. On signatures of knowledge. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 78–96. Springer, Berlin, Heidelberg, August 2006.
- [CP93] David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 89–105. Springer, Berlin, Heidelberg, August 1993.
- [CPY⁺] Stephen Curran, Artur Philipp, Hakan Yildiz, Sam Curren and Victor Martinez Jurado, Aritra Bhaduri, and Artem Ivanov. AnonCreds specification. <https://hyperledger.github.io/anoncreds-spec/>.
- [CPZ20] Melissa Chase, Trevor Perrin, and Greg Zaverucha. The Signal private group system and anonymous credentials supporting efficient verifiable encryption. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1445–1459. ACM Press, November 2020.

- [CS97] Jan Camenisch and Markus Stadler. Efficient group signature schemes for large groups (extended abstract). In Burton S. Kaliski, Jr., editor, *CRYPTO'97*, volume 1294 of *LNCS*, pages 410–424. Springer, Berlin, Heidelberg, August 1997.
- [CV02] Jan Camenisch and Els Van Herreweghen. Design and implementation of the idemix anonymous credential system. In Vijayalakshmi Atluri, editor, *ACM CCS 2002*, pages 21–30. ACM Press, November 2002.
- [DFKP16] Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, and Bryan Parno. Cinderella: Turning shabby X.509 certificates into elegant anonymous credentials with the magic of verifiable computation. In *2016 IEEE Symposium on Security and Privacy*, pages 235–254. IEEE Computer Society Press, May 2016.
- [Eur] European Council. European digital identity (eID): Council makes headway towards EU digital wallet, a paradigm shift for digital identity in europe. <https://www.consilium.europa.eu/en/press/press-releases/2022/12/06/european-digital-identity-eid-council-adopts-its-position-on-a-new-regulation-for-a-digital-wallet-at-eu-level/>.
- [FLM22] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. ZKAttest: Ring and group signatures for existing ECDSA keys. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 68–83. Springer, Cham, September / October 2022.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- [FYC24] Daniel Fett, Kristina Yasuda, and Brian Campbell. Selective Disclosure for JWTs (SD-JWT). Internet-Draft draft-ietf-oauth-selective-disclosure-jwt-14, Internet Engineering Task Force, November 2024. Work in Progress.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013.
- [GM17] Jens Groth and Mary Maller. Snarky signatures: Minimal signatures of knowledge from simulation-extractable SNARKs. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 581–612. Springer, Cham, August 2017.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [ide] iden3. Circom: Efficient circuit framework for programmable zero-knowledge. Webpage: <https://iden3.io/circom>, Source: <https://github.com/iden3/circom>.
- [KMSV21] Markulf Kohlweiss, Mary Maller, Janno Siim, and Mikhail Volkhov. Snarky ceremonies. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 98–127. Springer, Cham, December 2021.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010.
- [LKWL25] Tobias Looker, Vasilis Kalos, Andrew Whitehead, and Mike Lodder. The BBS Signature Scheme. Internet-Draft draft-irtf-cfrg-bbs-signatures-08, Internet Engineering Task Force, March 2025. Work in Progress <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/08/>.


```

{
  "typ": "JWT",
  "alg": "RS256",
  "kid": "e6q73nti-47exXmw_yS_-rGm-CFvy0sogtsEMAp0QS8"
}.{
  "email": "alice@contoso.com",
  "family_name": "Example",
  "given_name": "Alice",
  "login_hint": "0.aaaaabbbbbbbccccccddddddeeeeeeffffffggggggghhhhhiiiiijjjjjkkkkkkkllllllmmmmmmnnnnnnnnnooooooppqqrrrrrrssssssddd",
  "name": "Alice Example",
  "oid": "12345678-1234-abcd-1234-abcdef124567",
  "onprem_sid": "S-1-2-34-5678901234-1234567890-1234567890-1234567",
  "preferred_username": "alice@contoso.com",
  "rh": "0.aaaaabbbbbbbccccdddeeffff12345gggg12345_124_aaaaaa.",
  "sid": "12345678-1234-abcd-1234-abcdef124567",
  "sub": "aaabbbccccdddeeffffggggghhh123456789012",
  "upn": "alice@contoso.com",
  "uti": "AAABBBBccccddd1234567",
  "verified_primary_email": [
    "alice@contoso.com"
  ],
  "verified_secondary_email": [
    "alice2@contoso.com"
  ],
  "acct": 0,
  "aud": "relyingparty.example.com",
  "auth_time": 1733965568,
  "exp": 1736557568,
  "iat": 1733965568,
  "ipaddr": "203.0.113.0",
  "iss": "https://contoso.com",
  "jti": "fGYC01mK2dBTafCjGAoTQ",
  "nbf": 1733965568,
  "tenant_ctry": "US",
  "tenant_region_scope": "WW",
  "tid": "12345678-1234-abcd-1234-abcdef124567",
  "ver": "2.0",
  "xms_pdl": "NAM",
  "xms_tpl": "en"
}.[Signature]
}

```